

分布式多代理协作架构设计

版本：v3.0
日期：2026-05-01
定位：企业级 AI 代理集群的架构规范，以**专家化、权限归口、高稳定执行性**三大骨架贯穿代理分类、服务形态、权限体系、通信协议、协作模式、安全模型与企业应用

目录

- 1. [设计目标](#)
- 2. [代理分类体系](#)
- 3. [六种代理角色](#)
- 4. [四种服务形态](#)
- 5. [权限控制体系](#)
- 6. [系统架构图](#)
- 7. [通信层设计](#)
- 8. [协作模式](#)
- 9. [安全模型](#)
- 10. [代理生命周期管理](#)
- 11. [典型应用场景](#)
- 12. [与现有架构的差距与演进路径](#)
- 13. [附录A：消息协议定义](#)
- 14. [附录B：权限策略配置参考](#)

1. 设计目标

1.1 三大核心骨架

本架构以三个正交但互锁的核心原则为骨架，贯穿每一层设计：

骨架	含义	为什么
专家化		分散有多种维度：每人一个agent只见过该用户的问题，每领域一个agent只见过该领域的问题；集中则

骨架	含义	为什么
	一个 agent 集中服务多用户/多领域，见多识广，跨领域知识积累，越用越强	见过所有用户+所有领域的问题，才能发现跨用户、跨领域的关联
权限归口	所有外网请求→侦察代理唯一出口，所有内部数据→数据代理唯一访问归口，所有权限判定→主代理唯一决策点	分散=10个agent各自管权限=极易出现不一致；归口=权限逻辑只实现一次=一致性保证=审计集中
高稳定执行性	日常事务→固化技能处理（不依赖大模型推理，毫秒级），专家推理→大模型处理，两者共享记忆协同工作	固化技能保证日常事务的高稳定执行（无推理不确定性）；多进程架构实现故障隔离（任一代理异常不影响其他）

1.2 衍生原则

原则	含义	来源骨架
网络分区感知	代理部署位置决定其网络可达性和信任等级，架构天然适配企业安全边界	权限归口
永久与临时分离	知识资产（永久代理）持续进化成为专家；执行资源（临时代理）按需调度、用完释放	专家化
推拉结合	永久代理可主动推送信号（监控型）；临时代理被动响应指令（执行型）	永久与临时分离
决策归口	主代理是唯一的决策出口，子代理只提供输入和建议	权限归口
全链路可审计	代理间所有通信和操作写入不可篡改的审计日志	权限归口
权限即架构	权限不是附加功能，是每一层（专家化、归口、协作、数据）的横切骨架	权限归口
双轨执行	固化技能（确定性）与专家推理（概率性）并行，共享记忆，故障隔离	高稳定执行性

1.3 设计约束

- 主代理不可直接访问外网
- 内部数据不向 DMZ 方向流出原始数据（数据代理返回结果给核心区，经脱敏后才可跨区传输）
- 外部数据必经清洗和可信度标注
- 跨区通信必须经过认证和加密
- 临时代理无持久化、无记忆、无网络直连

- 权限变更热加载，不中断服务
- 日常事务优先走固化技能，不占用大模型推理资源

2. 代理分类体系

2.1 按生命周期

维度	永久代理	临时代理
本质	组织的知识资产，持续进化成为专家	项目的人力资源，用完释放
类比	全职员工	外包/顾问
知识	持续积累，跨领域关联，越用越强	无跨任务记忆
部署	常驻进程，跨网络分区	按需调度，无网络直连
通讯	可主动推送（告警/信号）	仅被动响应（任务→结果）
故障	健康检查+自动恢复	超时终止+主代理决定重试
典型	主代理、侦察代理、数据代理、专家代理	执行代理、验证代理

永久代理的专家化定位：永久代理之所以"永久"，不是为了节省启动开销，是因为**知识积累需要时间**。一个侦察代理监控了1000个信号源，它才知道什么是"异常"；一个财经专家分析了500份研报，它才能发现跨行业关联。临时代理用完就销毁，没有跨任务积累，无法成为专家——这是设计使然，不是缺陷。

2.2 按网络分区

分区	信任等级	代理	网络可达
核心区	L0-L2	主代理(L0)、数据代理(L1)、专家代理(L2)	内网全通，外网不通
DMZ	L3	侦察代理(L3)，无论永久/临时模式均共享DMZ部署和出站策略	仅出站HTTPS，不知内网结构
无网络	L4	执行代理、验证代理	无直连网络，仅处理注入的上下文

2.3 按能力模式

模式	特征	适用场景	响应时间	可靠性
固化技能	确定性执行，不依赖大模型推理	定时提醒、日程同步、账单跟踪、格式转换、L1自动审批	毫秒级	极高（无推理不确定性）
专家推理	概率性推理，依赖大模型	投研分析、架构评审、合同审查、L3深度审批	秒级	依赖模型质量

双轨协同：固化技能和专家推理共享记忆（MEMORY.md/知识库），固化技能处理不了的请求自动升级到专家推理，专家推理的结论可以固化为新技能。

用户请求 → 固化技能匹配？

- └ 是 → 固化技能执行（毫秒级，高稳定）
- └ 否 → 专家Agent推理（秒级，高智能）

↓ 结论验证通过

→ 固化为新技能（下次毫秒级处理）

3. 六种代理角色

3.1 🧠 主代理（Brain）— 永久 · 核心区

职责：意图理解、任务编排、决策、跨代理协调、权限执行

核心能力： - 接收用户请求，匹配十大主题分类，确定认知推理范式 - 拆解任务为子任务，选择合适的代理和协作模式 - 整合子代理结果，做出最终决策 - 管理永久话题（知识体系、信息收集、前沿追踪、技能体系） - **执行权限策略：**根据用户角色和群配置，决定允许/拒绝操作 - **权限归口的决策点：**所有权限判定（允许/拒绝访问）由此发出，子代理不自行决定 - **注意：**权限判定 ≠ 专业判断。权限判定（允许/拒绝访问）归口主代理；专业判断（合规/风险/估值）由专家提供，主代理综合决策

硬约束： - 不直接访问外网 - 不直接操作数据库 - 所有决策必须可追溯（记录推理链） - 权限检查先于任务执行

接口：

输入：用户请求 / 子代理上报（结果、告警、心跳）

输出：最终响应 / 任务下发 / 策略调整 / 权限拒绝

3.2 🚢 侦察代理 (Scout) — 永久/临时 • DMZ

职责：外部信息采集、情报收集、环境感知

两种运行模式：

模式	触发	行为	类比
永久模式	系统启动	持续监控 RSS/API/网站，发现信号主动上报	哨兵
临时模式	主代理指令	执行特定搜索任务，返回结果后待命	斥候

核心能力：- 多源信息采集（网页、API、RSS、社交媒体）- 采集内容自动标注：来源、时间、可信度、相关性 - 信号检测：异常波动、新出现的关键词、模式变化 - 主动推送：发现高价值信号时通过告警通道立即上报

硬约束：- 唯一外网出口：所有外网请求必须经此代理，这是权限归口的基础 - 不知内网结构 - 不存储内部数据 - 所有采集结果必须带可信度评分 - 单次采集结果大小受限（防数据外泄通道）

3.3 🏠 数据代理 (Vault) — 永久 • 核心区

职责：内部数据访问、知识库管理、数据治理

核心能力：- 读取内部数据库、知识库、文档系统、代码仓库 - 数据脱敏：根据请求方身份和权限等级决定返回完整数据还是脱敏版本 - 数据质量守门：脏数据不入库，可疑数据标注风险 - 访问审计：记录谁查了什么、什么时候查的

硬约束：- 唯一内部数据访问归口：所有内部数据访问必须经此代理，这是权限归口的基础 - 不修改数据源（只读，写入需主代理审批） - 不向 DMZ 方向推送原始数据 - 查询结果必须脱敏后才能发送给核心区以外的代理 - 数据访问遵循最小权限原则

3.4 🧑 专家代理 (Expert) — 永久 • 核心区

职责：领域深度分析、专业判断、知识演进

专家类型：

专家	领域	核心技能	典型输出
财经分析师	投研/宏观/行业	估值建模、研报解读、风险预警	投资建议、风险报告
专利分析师	专利/知识产权	检索、相似度、侵权检测、价值评估	侵权预警、专利地图
技术架构师	架构/工程	技术选型、代码审查、性能诊断	架构评审、优化方案
合规官	法务/监管	政策解读、合规检查、风险评估	合规报告、整改建议

自我改进机制： - 分析准确率追踪 → 策略调整 - 用户反馈（采纳/拒绝） → 知识库更新 - 跨领域关联发现 → 扩展知识边界 - 定期知识库审查 → 淘汰过时知识

硬约束： - 不直接访问数据源（通过数据代理获取）—— 权限归口 - 不直接访问外网（通过侦察代理获取）—— 权限归口 - 专业判断必须附带置信度和证据链

3.5 ⚙️ 执行代理（Worker） — 临时 • 按需调度

职责： 具体编码、文档生成、数据处理等执行任务

硬约束： - 无记忆：不持有跨任务状态 - 无持久化：不写入任何永久存储 - 无网络：不直接联网 - 受限时：必须在规定时间内完成 - 权限受限：只能访问本次任务上下文中声明的资源

3.6 ✅ 验证代理（Auditor） — 临时 • 按需调度

职责： 交叉验证、质量审计、合规检查

验证维度：

维度	检查内容
事实准确性	数据是否与来源一致，计算是否正确
逻辑一致性	推理链是否完整，结论是否从前提得出
合规性	是否违反安全策略、数据保护规则
完整性	是否有遗漏、是否覆盖所有要求
权限合规	操作是否在权限范围内

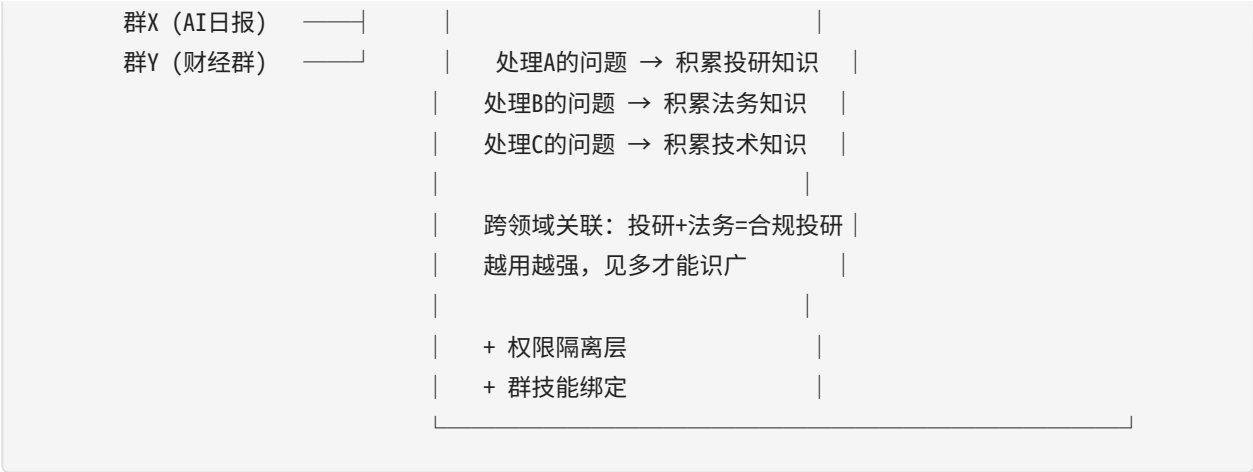
硬约束： - 独立于执行链，由主代理直接调度 - 审计日志不可篡改 - 验证失败必须给出具体原因和修复建议

4. 四种服务形态

4.1 形态一：单代理专家化服务（1:N）

一个 agent 集中服务多个用户/群——本质是专家化，不是共享。





为什么集中，而不是分散？

分散有多种维度，每种都不如集中：

分散维度	每个agent看到什么	看不到什么	结果
每人一个agent	只见过该用户的问题	其他用户的问题、跨用户关联	10个用户=10个互不相识的浅薄agent
每领域一个agent	只见过该领域的问题	其他领域的问题、跨领域关联	投研agent不知道法务agent的合同风险
集中（1:N专家agent）	所有用户+所有领域的问题	—	见多识广，越用越强

核心本质：见多识广

用户A问：XX行业趋势如何？
→ 专家agent处理 → 积累行业知识

用户B问：XX合同条款有无法律风险？
→ 专家agent处理 → 积累法务知识
→ 同时发现：A的行业趋势中提到的政策变化，正是B的合同条款风险的根因
→ 跨领域洞察：主动告知A和B，"你们的问题其实是同一个"

每人一个agent → A的agent不知道B的问题，B的agent不知道A的行业
每领域一个agent → 投研agent不知道法务的合同风险，法务agent不知道行业的政策变化
集中一个专家agent → 所有问题都经过它，配合知识管理机制，可以发现跨用户、跨领域的关联

权限隔离：让同一个专家对不同的人表现出不同边界

层	机制	作用
	<code>restricted_channels</code> 命令过滤	受限群禁止执行命令、修改技能等有副作用操作

层	机制	作用
命令隔离		
工具隔离	<code>restricted_channels</code> 工具集过滤	受限群只允许只读工具（搜索、查询），禁止写操作
技能隔离	<code>channel_skill_bindings</code>	不同群绑定不同技能组合（AI群→AI技能，财经群→财经技能）

提示词隔离：

机制	作用	实现
<code>channel_prompts</code>	按群注入不同的系统提示	群X提示"你是一个AI分析师"，群Y提示"你是一个财经顾问"
群 SOUL.md	群级角色覆盖	<code>~/.hermes/groups/{chat_id}/SOUL.md</code>
群 MEMORY.md	群级记忆	<code>~/.hermes/groups/{chat_id}/MEMORY.md</code>

用户级隔离：

机制	作用	状态
<code>user_roles</code> (admin/trusted/blacklist)	用户能做什么	✅ 已实现
用户偏好画像	同群不同用户看到不同内容排序	❌ 待实现
会话级资源配额	防止单用户耗尽 agent 迭代预算	❌ 待实现

优势： - **专家化：**一个agent处理多领域问题，积累跨领域知识，越用越强 - **跨领域洞察：**分散agent做不到的关联发现 - **维护成本低：**一次维护所有用户受益 - 配置简单，权限变更即时生效

局限： - 单点瓶颈：agent 是单进程，高并发时排队 - 权限设计必须严谨：共享专家，必须确保A的知识不泄漏给B - 群记忆隔离需完善：当前共享 MEMORY.md，需群级隔离

4.2 形态二：多代理权限归口调度（N:1）

多个 agent 按需调度，服务于同一个用户/请求——本质是权限归口，不是流量分发。





为什么集中调度/数据访问，而不是每个agent自己管？

分散权限（每个agent自己管）	集中归口（调度/数据代理统一管）
10个agent=10个权限实现	权限逻辑只实现一次
极易出现不一致：A允许但B拒绝同样的操作	一致性保证：所有请求必经此点
审计分散：10个agent各自记录，难以关联	审计集中：一条链路完整可追溯
脱敏分散：每个agent自己判断脱敏策略	脱敏集中：数据代理统一执行，策略一致
新增agent必须重新实现权限逻辑	新增agent只需声明能力，权限由归口层控制

核心本质：权限归口

没有归口层：
Agent-A 直接访问数据库 → A自己判断权限 → A的权限逻辑可能有bug
Agent-B 直接访问外网 → B自己判断什么能抓 → B可能抓了不该抓的
Agent-C 直接返回数据 → C自己判断脱敏 → C可能忘了脱敏PII

有归口层：
所有数据请求 → 数据代理（唯一有DB权限的）→ 脱敏 → 返回
所有外网请求 → 侦察代理（唯一有外网权限的）→ 标注可信度 → 返回
所有权限判定 → 主代理（唯一有策略引擎的）→ 允许/拒绝 → 执行

权限逻辑只写一次，审计只有一个来源，脱敏只有一个策略。

调度策略：

策略	触发条件	路由目标	示例
复杂度路由	输入长度 < 160字符	快模型 Agent	"今天天气？" → Agent-A
主题路由	匹配十大主题分类	对应专家 Agent	"分析这份研报" → Agent-B
技能路由	请求触发特定技能	预装该技能的 Agent	"生成PPT" → Agent-C
负载路由	当前 Agent 负载 > 阈值	空闲 Agent	高峰期 → 备用 Agent
降级路由	Agent 响应超时	备用 Agent	Agent-A 超时 → Agent-D

现有基础：

能力	现有实现	差距
复杂度路由	<code>smart_model_routing</code> （按输入长度）	仅区分简单/复杂，不区分主题
主题路由	无	需意图分类器 + 主题→Agent 映射
技能路由	<code>channel_skill_bindings</code> （按群绑定）	需扩展为按请求动态绑定
负载路由	无	需 Agent 池 + 健康检查
会话连续性	同一 session 保持同一 Agent	跨 Agent 切换时需上下文传递

优势： - **权限归口：**所有权限逻辑集中实现，一致性保证 - **审计集中：**一条链路完整可追溯 - **脱敏统一：**数据代理统一执行，不会出现某个agent忘了脱敏 - **用户无感：**一个入口，后端自动选最优 Agent - **弹性伸缩：**高峰期扩容 Agent 池

局限： - 归口层是单点——需要高可用设计 - 跨 Agent 会话连续性需额外处理 - 归口层性能是系统瓶颈

4.3 形态三：多对多协作（N:N）

多个 agent 之间对等协作，没有固定主从。



适用场景：复杂任务需要多代理并行处理，如投研（侦察采集+数据查询+专家分析+验证审计）。

4.4 形态四：混合形态

实际部署往往是三种形态的组合。

用户群(1:N专家化) → 归口调度层(N:1权限归口) → 代理集群(N:N协作)

企业级部署几乎都是混合形态：用户侧1:N专家化，调度侧N:1权限归口，代理侧N:N协作。

5. 权限控制体系

5.1 权限模型总览

权限不是单一维度，而是**四个正交维度**的组合：

权限 = f(谁，在哪，做什么，访问什么数据)

谁 → 用户身份 (user_id) + 用户角色 (role)

在哪 → 上下文 (群/频道/会话) + 网络区 (核心/DMZ/外部)

做什么 → 操作类型 (命令/工具/技能/通信)

访问什么 → 数据范围 (公开/内部/机密/受限)

5.2 用户角色体系

角色定义

角色	权限范围	典型用户
admin	全部权限：命令执行、技能管理、配置变更、数据写入、用户管理	管理员
trusted	大部分权限：问答、搜索、工具使用、数据读取；禁止配置变更和用户管理	高级用户
blacklist	禁止交互	被封禁用户

角色继承与覆盖

全局默认角色 (config.yaml: gateway.user_roles)

↓ 群级覆盖

群角色 (restricted_channels.{chat_id}.users[].role)

↓ 会话级覆盖

临时提权/降权 (管理员手动操作)

优先级：会话级 > 群级 > 全局级

角色能力矩阵

能力	admin	trusted	blacklist
问答/对话	✓	✓	✗

能力	admin	trusted	blacklist
搜索/查询	✓	✓	✗
工具执行（只读）	✓	✓	✗
工具执行（写入）	✓	✓(受限)	✗
技能管理	✓	✗	✗
命令执行	✓	✗	✗
配置变更	✓	✗	✗
用户管理	✓	✗	✗
数据写入	✓	✓(受限)	✗

5.3 群/频道级权限

三道防线（已实现）

防线	位置	作用	实现
命令过滤	gateway/run.py L4273	受限群禁止执行命令	检查 <code>restricted_channels</code> → 拦截 slash 命令
工具集过滤	gateway/run.py L10574	受限群限制可用工具	动态修改 <code>enabled_toolsets</code>
提示词过滤	gateway/session.py L377	受限群注入权限提示	在系统提示中告知 agent 当前权限边界

群权限配置

```
gateway:
  restricted_channels:
    oc_6ed1cc645057da3bbe93c6120219a61e: # AI日报群
  users:
    - id: ou_d366ba075912a3865430a7fe7020454b
      role: admin
  allowed_commands: [] # 空列表 = 禁止所有命令
  allowed_toolsets: # 允许的工具集
    - web
    - file
  denied_tools: # 明确禁止的工具
    - terminal
```

```
- write_file
- skill_manage
blacklist: []
```

群技能绑定

```
feishu:
channel_skill_bindings:
- id: "oc_6ed1cc645057da3bbe93c6120219a61e"
  skills: ["group-thinking", "source-evolution", "core-thinking"]
- id: "oc_finance_group_id"
  skills: ["financial-analysis", "source-evolution"]
```

群提示词注入

```
feishu:
channel_prompts:
"oc_6ed1cc645057da3bbe93c6120219a61e": |
  你是AI日报群的助手。权限规则：
   允许：回答问题、搜索信息、推送日报
   禁止：修改技能、执行命令、任何有副作用的操作
"oc_finance_group_id": |
  你是财经分析群的助手。专精：投研分析、宏观解读、风险预警。
```

5.4 代理间权限

信任分级

等级	代理	能做	不能做
L0 完全信任	主代理	一切	—
L1 高信任	数据代理	读取内部数据	修改数据源、外传原始数据
L2 中信任	专家代理	读取主代理下发的上下文	直接访问数据源、访问外网
L3 低信任	侦察代理	访问外网、返回结果	知晓内网结构、存储内部数据
L4 不信任	临时执行/验证代理	仅处理本次任务上下文	持久化、记忆、网络直连

代理间通信权限矩阵

发送方 \ 接收方	主代理	数据代理	专家代理	侦察代理	临时代理
主代理	—	✓	✓	✓	✓
数据代理	✓	—	✗	✗	✗
专家代理	✓	✗	⚠ (需主代理中转)	✗	✗
侦察代理	✓	✗	✗	—	✗
临时代理	✓	✗	✗	✗	—

规则： - 所有子代理→主代理：✓ 允许（结果回传、告警上报） - 子代理→子代理：✗ 默认禁止，需主代理中转 - 专家代理间通信：⚠ 需主代理编排（会诊模式），不可直接通信

令牌授权

```
{
  "token_id": "tk_20260501_001",
  "issuer": "brain",
  "subject": "scout-01",
  "task_id": "task_20260501_001",
  "permissions": {
    "channels": ["task.execute", "result.submit", "alert.signal"],
    "data_scope": "external_only",
    "tools": ["web_search", "web_extract", "browser_navigate"],
    "max_result_size_kb": 512,
    "expires_at": "2026-05-01T10:05:00Z"
  }
}
```

5.5 数据权限

数据分级

级别	标签	可流向	脱敏要求
Public	public	所有区域	无
Internal	internal	仅核心区	跨区传输时需脱敏
Confidential	confidential	仅主代理+数据代理	必须脱敏后才可转发给专家
Restricted	restricted	仅主代理	不可转发

级别	标签	可流向	脱敏要求
PII	pii	必须脱敏	脱敏后才可传输（任何区域）

脱敏规则

请求方	数据分级	返回策略
主代理	任何	完整数据
数据代理	internal+	完整数据（自身职责）
专家代理	internal	脱敏后返回（去除 PII 和 restricted 字段）
侦察代理	public	仅 public 数据
临时代理	按任务令牌	仅令牌声明的数据范围

5.6 权限变更热加载

设计原则：权限变更即时生效，不中断服务，不重建会话。

变更类型	触发方式	生效时机	影响范围
用户角色变更	修改 config.yaml	下次请求时	该用户的所有会话
群权限变更	修改 restricted_channels	下次请求时	该群的所有会话
技能绑定变更	修改 channel_skill_bindings	下次新会话	该群的新会话（旧会话保持缓存）
代理信任等级变更	证书更新	mTLS 重连时	该代理的所有连接
数据分级变更	策略引擎更新	下次查询时	所有后续查询

注意：系统提示（含权限提示）在会话开始时冻结为快照（_system_prompt_snapshot），会话内权限变更不修改已缓存的系统提示，但命令/工具过滤在每次请求时实时检查。

6. 系统架构图

6.1 总体架构（三大骨架）

```
graph TB
    subgraph USERS["用户/群"]
        U1["👤 用户A (admin)"]
        U2["👤 用户B (trusted)"]
        U3["👤 群X (restricted)"]
        U4["👤 群Y (开放)"]
    end

    subgraph L1["Layer 1: 专家化层 (1:N)"]
        PERM["🔑 权限策略引擎<br/>user_roles + restricted_channels<br/>channel_skill_bindings"]
        EXPERT_AGENT["🧑 专家 Agent<br/>见多识广 · 跨领域知识积累<br/>+ 权限隔离 · 群技能绑定"]
    end

    subgraph L2["Layer 2: 权限归口层 (N:1)"]
        GATE["🚪 权限归口调度<br/>唯一决策点 · 唯一出口 · 唯一入口"]
        AGENT_A["Agent-A<br/>快模型/固化技能"]
        AGENT_B["Agent-B<br/>强模型/专家推理"]
        AGENT_C["Agent-C<br/>领域专家"]
    end

    subgraph L3["Layer 3: 协作层 (N:N)"]
        BUS["📡 消息总线<br/>task/result/alert/hb/audit"]
        ORCHESTRA["🎭 编排引擎"]
        REGISTRY["📖 服务注册"]
    end

    subgraph AGENTS["代理集群"]
        BRAIN["🧠 主代理<br/>唯一决策点"]
        SCOUT["🕵️ 侦察代理<br/>唯一外网出口"]
        VAULT["🏠 数据代理<br/>唯一数据访问归口"]
        EXPERT["🧑 专家代理"]
        WORKER["⚙️ 执行代理(临时)"]
        AUDITOR["✅ 验证代理(临时)"]
    end

    subgraph DATA["数据层"]
        DB["🗄️ (内部数据库)"]
        KB["📚 (知识库)"]
    end

    subgraph EXT["外部"]
        WEB["🌐 互联网"]
    end
```



```

end

U1 & U2 & U3 & U4 -->|请求| PERM
PERM -->|鉴权| EXPERT_AGENT
EXPERT_AGENT -->|路由| GATE
GATE --> AGENT_A & AGENT_B & AGENT_C
AGENT_A & AGENT_B & AGENT_C --> BUS
BUS --> ORCHESTRA
ORCHESTRA --> BRAIN & SCOUT & VAULT & EXPERT & WORKER & AUDITOR
SCOUT -->|唯一出口| WEB
VAULT -->|唯一入口| DB & KB

style L1 fill:#1a1a2e,stroke:#e94560,color:#fff
style L2 fill:#16213e,stroke:#0f3460,color:#fff
style L3 fill:#0f3460,stroke:#533483,color:#fff
style AGENTS fill:#2d2d2d,stroke:#ffd700,color:#fff

```

6.2 网络分区与数据流向

```

graph LR
    subgraph CORE["核心区 — 信任 L0/L1/L2"]
        BRAIN["🧠 主代理<br/>唯一决策点"]
        VAULT["🏠 数据代理<br/>唯一数据访问归口"]
        EXPERT["👩 专家"]
    end

    subgraph DMZ["DMZ — 信任 L3"]
        SCOUT["📡 侦察代理<br/>唯一外网出口"]
    end

    subgraph EXT["外部 — 信任 L4"]
        EXT_DATA["外部数据"]
    end

    subgraph DATA["数据层 — 信任 L0"]
        INTERNAL["内部数据"]
    end

    BRAIN -->|"mTLS + 令牌"| SCOUT
    SCOUT -->|"可信度标注"| BRAIN
    BRAIN -->|"查询"| VAULT
    VAULT -->|"完整结果"| BRAIN
    BRAIN -->|"上下文+请求"| EXPERT
    EXPERT -->|"分析+置信度"| BRAIN
    SCOUT -->|"仅出站HTTPS"| EXT_DATA
    INTERNAL -->|"数据读取"| VAULT

```

```
SCOUT -. ->|"🚫 禁止"| INTERNAL
EXPERT -. ->|"🚫 禁止"| EXT_DATA
```

```
style CORE fill:#1a1a2e,stroke:#e94560,color:#fff
style DMZ fill:#16213e,stroke:#0f3460,color:#fff
style DATA fill:#0f3460,stroke:#533483,color:#fff
style EXT fill:#533483,stroke:#e94560,color:#fff
```

6.3 权限体系架构

```
graph TB
    subgraph INPUT["请求入口"]
        USER_REQ["用户请求<br/>user_id + chat_id + message"]
    end

    subgraph AUTH["权限策略引擎"]
        USER_ROLE["👤 用户角色判定<br/>admin/trusted/blacklist"]
        CHAN_PERM["👥 群权限判定<br/>restricted_channels"]
        DATA_CLASS["📁 数据分级判定<br/>public/internal/confidential/restricted"]
        AGENT_TRUST["🛡️ 代理信任等级<br/>L0-L4"]
    end

    subgraph ENFORCE["执行层"]
        CMD_FILTER["命令过滤<br/>允许/拒绝 slash 命令"]
        TOOL_FILTER["工具过滤<br/>允许/拒绝工具调用"]
        SKILL_BIND["技能绑定<br/>channel_skill_bindings"]
        PROMPT_INJECT["提示词注入<br/>channel_prompts + 权限边界"]
        DATA_MASK["数据脱敏<br/>按请求方等级返回"]
    end

    subgraph AUDIT_LAYER["审计层"]
        AUDIT["📝 审计日志<br/>不可篡改 • 可追溯"]
    end

    USER_REQ --> USER_ROLE --> CMD_FILTER
    USER_REQ --> CHAN_PERM --> TOOL_FILTER
    USER_REQ --> CHAN_PERM --> SKILL_BIND
    USER_REQ --> CHAN_PERM --> PROMPT_INJECT
    AGENT_TRUST --> DATA_CLASS --> DATA_MASK
    CMD_FILTER & TOOL_FILTER & SKILL_BIND & PROMPT_INJECT & DATA_MASK --> AUDIT

    style AUTH fill:#1a1a2e,stroke:#e94560,color:#fff
    style ENFORCE fill:#16213e,stroke:#0f3460,color:#fff
    style AUDIT_LAYER fill:#0f3460,stroke:#533483,color:#fff
```

6.4 永久代理生命周期

stateDiagram-v2

```
[*] --> Initializing: 系统启动
Initializing --> Ready: 加载知识库+注册通道+心跳上线
Ready --> Active: 收到任务/信号
Active --> Processing: 执行任务
Processing --> Reporting: 生成结果
Reporting --> Learning: 沉淀知识
Learning --> Ready: 知识入库
Active --> Monitoring: 无任务(侦察代理)
Monitoring --> Alerting: 发现信号
Alerting --> Active: 上报主代理
Ready --> Evolving: 定期触发
Evolving --> Ready: 进化完成
Ready --> Shutdown: 系统停止
Shutdown --> [*]
```

6.5 临时代理生命周期

stateDiagram-v2

```
[*] --> Spawned: 主代理创建
Spawned --> ContextLoading: 注入上下文+令牌
ContextLoading --> Executing: 开始执行
Executing --> Completed: 任务完成
Executing --> Failed: 执行失败
Executing --> Timeout: 超时
Completed --> ResultReturn: 返回结果
Failed --> ResultReturn: 返回错误
Timeout --> ResultReturn: 返回超时
ResultReturn --> Destroyed: 销毁
Destroyed --> [*]
```

7. 通信层设计

7.1 协议栈

层	技术选型	说明
传输层	HTTP/2 + SSE 或 WebSocket	

层	技术选型	说明
		HTTP/2 多路复用；SSE 流式输出；WebSocket 双向实时
序列化	JSON-RPC 2.0	标准化请求/响应；MessagePack 可选（高性能）
安全	mTLS 双向认证 + JWT 令牌	代理间双向证书验证；令牌控制细粒度权限
可靠	至少一次投递 + 幂等处理	消息 ID 去重；死信队列处理异常
消息中间件	Redis Stream（轻量）或 NATS（集群）	Redis Stream 单集群；NATS 跨数据中心

7.2 消息格式

任务下发

```
{
  "jsonrpc": "2.0",
  "method": "task.execute",
  "id": "task_20260501_001",
  "params": {
    "task_id": "task_20260501_001",
    "target_agent": "scout-01",
    "task_type": "web_research",
    "goal": "搜索 XX 领域最新论文",
    "context": "用户正在进行投研分析...",
    "timeout_seconds": 300,
    "priority": "normal",
    "callback_channel": "result.task_20260501_001",
    "credentials_scope": ["web_search", "web_extract"],
    "created_at": "2026-05-01T10:00:00Z",
    "created_by": "brain"
  }
}
```

告警消息（侦察代理主动推送）

```
{
  "jsonrpc": "2.0",
  "method": "alert.signal",
  "id": "alert_20260501_001",
  "params": {
    "alert_id": "alert_20260501_001",
    "agent_id": "scout-01",
  }
}
```

```
"alert_type": "breaking_event",
"severity": "high",
"topic": "AI前沿追踪",
"signal": "DeepSeek 发布新模型",
"source": "https://...",
"confidence": 0.72,
"suggested_action": "交专家代理评估",
"detected_at": "2026-05-01T09:58:00Z"
}
}
```

7.3 通道 QoS 策略

通道	投递保证	持久化	重试	超时
task.*	至少一次	是	3次，指数退避	30s确认
result.*	至少一次	是	3次	按任务设定
alert.*	尽力投递	否	0次	即时
hb.*	尽力投递	否	0次	5s
audit.*	至少一次+去重	是，不可篡改	指数退避至上限	24h超时

8. 协作模式

8.1 模式一：咨询型（1:N 专家化 + 单 agent）

最常见模式，单 agent 通过权限隔离服务多个用户。

```
sequenceDiagram
    participant UA as 用户A (admin)
    participant UB as 用户B (trusted)
    participant B as 🧠 主代理
    participant S as 🕵️ 侦察代理
    participant V as 🏠 数据代理

    UA->>B: "分析行业趋势" (admin: 允许工具执行)
    UB->>B: "查一下新闻" (trusted: 允许搜索)
    B->>B: 权限检查 → A可执行, B只读
    B->>S: task: 搜索外部信息 (唯一出口)
    B->>V: task: 查询内部数据 (唯一入口)
```

```
S-->>B: result
V-->>B: result (对B脱敏)
B->>UA: 完整分析 (含工具执行结果)
B->>UB: 搜索结果 (脱敏版)
```

8.2 模式二：调度型（N:1 权限归口 + 多 agent 服务同一用户）

调度器不是简单的流量路由，而是**权限归口层**——所有请求经调度器鉴权后分发，所有结果经调度器脱敏后返回。

```
sequenceDiagram
    participant U as 用户
    participant G as 🏠 权限归口调度器
    participant A as Agent-A (快)
    participant B as Agent-B (强)
    participant C as Agent-C (专家)
    participant S as 🕵️ 侦察代理
    participant V as 🏢 数据代理

    U->>G: "简单问题"
    G->>G: 权限归口：鉴权+数据范围判定
    G->>G: 复杂度路由 → Agent-A
    G->>A: 转发请求（注入权限令牌）
    A-->>G: 快速回答
    G-->>U: 响应（审计记录）

    U->>G: "深度分析"
    G->>G: 权限归口：鉴权+请求外部数据
    G->>S: task: 搜索外部信息（唯一出口）
    S-->>G: result（可信度标注）
    G->>G: 主题路由 → Agent-B
    G->>B: 转发请求（注入脱敏上下文）
    B-->>G: 深度分析
    G-->>U: 响应（脱敏+审计）

    U->>G: "专利侵权分析"
    G->>G: 权限归口：鉴权+请求内部数据
    G->>V: task: 查询内部专利库（唯一入口）
    V-->>G: result（脱敏版）
    G->>G: 技能路由 → Agent-C
    G->>C: 转发请求（注入脱敏上下文）
    C-->>G: 专家分析
    G-->>U: 响应（脱敏+审计）
```

8.3 模式三：监控推送型（永久代理主动发现）

```
sequenceDiagram
    participant S as 🕵️ 侦察代理
    participant B as 🧠 主代理
    participant E as 🧑 专家代理
    participant U as 用户

    loop 持续监控
        S->>S: 扫描信息源
    end

    S->>B: alert: 发现异常信号
    B->>B: 权限检查: 该信号属于哪个群?
    B->>E: task: 评估信号
    E-->>B: result: 评估报告
    B->>U: 推送(按群权限决定推送内容)
    Note over B,U: admin→完整报告<br/>trusted→摘要+建议
```

8.4 模式四：项目型（临时团队）

```
sequenceDiagram
    participant B as 🧠 主代理
    participant W1 as ⚙️ 执行代理A
    participant W2 as ⚙️ 执行代理B
    participant A as ✅ 验证代理

    B->>B: 权限检查: 用户是否有写入权限?
    B->>W1: task: 编码实现 (注入令牌)
    B->>W2: task: 编写测试 (注入令牌)
    W1-->>B: result: 代码
    W2-->>B: result: 测试
    B->>A: task: 交叉验证
    A-->>B: result: 通过/失败
    Note over B,A: 临时代理销毁
```

8.5 模式五：会诊型（多专家 N:N 协作）

```
sequenceDiagram
    participant B as 🧠 主代理
    participant E1 as 🧑 财经专家
    participant E2 as 🧑 技术专家
    participant E3 as 🧑 合规官
```

B->>E1: task: 市场角度（数据脱敏后下发）
 B->>E2: task: 技术角度
 B->>E3: task: 合规角度
 E1-->>B: 看多
 E2-->>B: 可行
 E3-->>B: 需注意X
 B->>B: 识别共识(可行)与分歧(风险)
 B->>B: 按用户角色决定输出深度

8.6 模式六：链式加工型

```
sequenceDiagram
    participant S as 🕵️ 侦察代理
    participant B as 🧠 主代理
    participant V as 🏠 数据代理
    participant E as 🧑 专家代理
    participant A as ✅ 验证代理

    S->>B: result: 原始数据+可信度
    B->>V: task: 清洗+结构化
    V-->>B: result (脱敏级别按下游需求)
    B->>E: task: 深度分析
    E-->>B: result: 分析报告
    B->>A: task: 审计验证
    A-->>B: result: 验证通过
    B->>B: 按用户权限决定输出内容
```

9. 安全模型

9.1 认证与授权

机制	范围	实现
mTLS 双向认证	代理间通信	每个代理持有唯一证书，核心区 CA 签发
JWT 令牌	任务级授权	主代理签发短期令牌（5分钟），含操作/数据/时间范围
用户角色	用户级	admin/trusted/blacklist
群权限	群级	restricted_channels + channel_skill_bindings
数据分级	数据级	public/internal/confidential/restricted/pii

9.2 数据保护

规则	实现
内部数据只出不进	数据代理不向 DMZ 推送原始数据
外部数据必经清洗	侦察代理采集内容必须标注来源和可信度
跨区通信必经审计	所有消息写入审计日志
脱敏分级	按请求方信任等级返回不同粒度数据
结果大小限制	侦察代理单次返回不超过上限

9.3 审计系统

审计日志仅追加、不可篡改（WORM），保留 365 天。支持按代理、时间、任务ID 检索。异常模式自动检测。

9.4 权限归口即安全架构

权限归口不仅是权限管理的便利手段，它本身就是安全架构的核心：

维度	分散权限	权限归口
攻击面	N个agent各自暴露N个访问入口，攻击面分散但难以统一防护	1个出口/入口/决策点=安全策略一致+审计集中（但单点价值更高，需更强的防护）
审计	N个审计来源，难以关联，容易出现审计盲区	1条链路完整可追溯，无审计盲区
一致性	N个权限实现，极易出现不一致（A允许但B拒绝）	1个权限实现，一致性保证
脱敏	N个脱敏策略，某个agent忘了脱敏就是安全事故	1个脱敏策略，数据代理统一执行
凭证管理	N个agent持有凭证，N个泄漏风险	凭证仅归口层持有，1个管控点
故障影响	某个agent权限bug→安全漏洞	归口层bug可集中修复，影响范围可控

结论：权限归口把安全从"N点防御"变为"1点防御"，不是简化了安全，而是**集中了安全**。分散权限的"多点防御"听起来更强，实际上是"多点漏洞"——只要有一个点的实现有bug，整个系统就不安全。

10. 代理生命周期管理

10.1 永久代理自我改进

机制	触发	行动
准确率追踪	每次分析被验证后	记录对/错，计算准确率趋势
反馈学习	用户采纳/拒绝建议	采纳→强化；拒绝→分析原因→调整
知识淘汰	每周	审查知识库，标记过时/矛盾条目
跨域关联	发现新知识时	检查与其他领域知识的关联
策略进化	准确率持续低于阈值	调整分析策略、权重、优先级

专家化积累：永久代理的核心价值不是"一直运行"，而是"一直学习"。处理100次投研请求→积累行业模式，处理50次合同审查→积累条款风险点，当两种知识交叉→发现"这个行业的合同条款有系统性风险"。这就是专家化——见多识广。

10.2 健康检查与故障恢复

检查项	频率	超时处理
心跳	30s	3次缺失→标记不可用→告警
任务执行	按任务超时	超时→终止→主代理决定重试或换路
知识库一致性	每小时	不一致→从备份恢复→告警
通信通道	每 10s	断连→指数退避重连→上限后告警

故障隔离：任一代理异常不影响其他代理继续执行。侦察代理挂了→主代理暂用缓存数据+告警；数据代理挂了→主代理降级为无内部数据模式；专家代理挂了→主代理用基础推理兜底。核心原则：**部分降级，而非全局失败。**

10.3 双轨执行：固化技能 vs 专家推理

维度	固化技能	专家推理
执行引擎	确定性规则/脚本	大模型推理
响应时间	毫秒级	秒级
可靠性	极高（无推理不确定性）	依赖模型质量

维度	固化技能	专家推理
适用场景	定时提醒、日程同步、账单跟踪、格式转换、L1自动审批	投研分析、架构评审、合同审查、L3深度审批
知识来源	技能定义 + 配置	知识库 + 上下文推理
故障模式	规则匹配失败→升级到专家推理	推理超时/质量不足→降级或告警

协同机制：

请求到达 → 固化技能匹配

- └─ 匹配成功 → 固化技能执行（毫秒级）
 - └─ 执行失败 → 自动升级到专家推理
- └─ 匹配失败 → 专家Agent推理（秒级）
 - └─ 推理结论验证通过 → 可固化为新技能
→ 下次同类请求毫秒级处理

示例演进路径：

第1次采购审批请求 → 专家Agent推理（秒级，全流程分析）

第10次同类请求 → 专家推理效率提升（模式匹配加速）

第50次同类请求 → 固化为L1自动审批技能（毫秒级，规则判定）

→ 越用越快，越用越稳

11. 典型应用场景

每个场景标注：**核心价值**（为什么需要这个架构）、**服务形态**（→ch4）、**协作模式**（→ch8）、**权限要点**（→ch5）

按三层组织：**个人层** → **团队层** → **企业层**

个人层

11.1 个人智能助理

核心价值：一个专家agent汇聚个人全部事务和信息——日程、财务、健康、社交、知识——跨领域关联发现（ch4.1专家化），同时安全归口保证个人隐私不泄漏（ch4.2权限归口）

服务形态：1:N 专家化 (→ch4.1)

协作模式：咨询型 + 监控推送型 (→ch8.1, ch8.3)

个人用户 → 专家Agent(1:N, 一个人全部事务的归口)

日程管理 → "明天上午有会, 下午要交报告"

财务提醒 → "信用卡本周五还款日, 余额够吗?"

健康跟踪 → "你这周运动了2天, 比上周少1天"

知识归档 → "你上周读的那篇文章, 跟今天这个新闻有关联"

→ 跨领域洞察 (专家化价值):

日程+财务: "你要出差的那天, 正好是账单截止日, 提前设置自动还款?"

健康+日程: "你下周会议密集, 建议在早间插入运动时间"

知识+工作: "你收藏的那篇论文的方法, 正好可以解今天代码的问题"

安全归口 (→ch5.4代理间权限, ch9.4权限归口即安全):

所有个人数据 → 只有一个数据代理持有, 其他代理不直接访问

侦察代理(唯一外网出口) → 出行/天气/新闻等外部信息, 审计所有外网请求

数据代理(唯一数据访问归口) → 银行/日历/健康API等个人数据, 统一脱敏后注入上下文

个人凭证 (银行token/日历API key) → 仅数据代理持有, 其他代理无法触碰

高稳定执行性 (→ch10.3双轨执行):

日常事务不依赖大模型推理 → 固化技能处理 (定时提醒、账单跟踪、日程同步)

专家Agent处理需要理解的请求 → 与固化技能共享记忆, 协同工作

侦察代理7×24运行 → 监控航班变动、天气预警、关注领域新闻

故障隔离 (→ch10.2) → 任何一个代理异常不影响其他代理继续执行

权限要点 (→ch5.2用户角色, ch5.5数据分级):

用户本人 → 完整数据+操作权限

家庭成员(trusted) → 共享日程+共享账单, 不暴露私人通讯/健康

外部服务 → 零数据泄漏, 所有API调用经数据代理脱敏

审计日志 → 用户可查"agent替我做了什么、访问了什么"

11.2 个人驾驶舱

核心价值：个人助理的延伸——从"被动响应"升级为"主动一屏掌控", 我的日程/任务/知识/健康/财务一目了然

服务形态：1:N 专家化 (→ch4.1)

协作模式：咨询型 + 监控推送型 (→ch8.1, ch8.3)

个人驾驶舱 = 个人助理的仪表盘:

我的日程/任务/知识/健康/财务 → 一屏掌控

专家Agent理解我的上下文 → 异常主动推送

例: "你本周3个deadline撞车了, 建议调整优先级"

数据流 (→ch6.2网络分区):

数据代理(唯一数据访问归口) → 汇聚个人全部数据 → 按驾驶舱视图结构化返回

侦察代理(唯一外网出口) → 监控外部变化(天气/航班/新闻) → 异常推送
固化技能 → 定时刷新驾驶舱数据(不依赖大模型推理)

权限要点:

仅用户本人可见 → 最严格的PII级别(→ch5.5数据分级)

家庭共享视图 → 数据代理脱敏后返回(去除私人通讯/健康细节)

团队层

11.3 团队驾驶舱

核心价值: 团队成员负载/协作瓶颈/知识共享度 → 团队健康度, 专家Agent积累团队模式发现协作问题(ch4.1专家化)

服务形态: 1:N 专家化(→ch4.1)

协作模式: 咨询型 + 监控推送型(→ch8.1, ch8.3)

团队驾驶舱:

团队成员负载/协作瓶颈/知识共享度 → 团队健康度

专家Agent积累团队模式 → 发现协作问题

例: "A和B都在等C的接口, C已超载, 建议D协助"

数据流(→ch5.4代理间权限):

数据代理 → 唯一内部数据访问归口, 按驾驶舱角色脱敏

团队驾驶舱 → 返回团队汇总, 不暴露个人薪酬等敏感信息

侦察代理 → 唯一外网出口, 监控外部风险信号

验证代理 → 审计驾驶舱数据准确性

权限要点:

团队Lead → 看团队汇总+个人负载(不含薪酬等敏感信息)

成员 → 看自己的任务+团队公开信息

11.4 项目驾驶舱

核心价值: 项目进度/风险/资源/里程碑 → 项目状态全景, 数据代理归口多项目数据发现跨项目资源冲突(ch4.2权限归口)

服务形态: 1:N 专家化 + N:1 权限归口(→ch4.1, ch4.2)

协作模式: 咨询型 + 监控推送型(→ch8.1, ch8.3)

项目驾驶舱:

项目进度/风险/资源/里程碑 → 项目状态全景

数据代理归口多项目数据 → 跨项目资源冲突发现

例："项目X占用3个前端，项目Y下周需要2个，资源冲突"

权限要点：

PM → 看项目全景+资源占用

成员 → 看自己参与的项目的进度+任务

管理层 → 跨项目资源汇总（→ch5.5数据分级：跨项目数据为confidential级别）

11.5 智能审批（取代电子流）

核心价值：传统电子流是"表单+路由"，智能审批是"理解+判断+执行"——agent理解审批意图、自动核验合规性、关联历史决策、执行审批结果，从流程驱动变为智能驱动

服务形态：1:N 专家化 + N:N 协作（→ch4.1, ch4.3）

协作模式：链式加工型 + 咨询型（→ch8.6, ch8.1）

传统电子流的问题：

申请人填表 → 固定路由到审批人 → 审批人手动审查 → 通过/驳回

❌ 流程僵化：紧急采购走标准流程，3天批不下来

❌ 信息割裂：审批人看不到关联信息（预算余额、历史同类审批、合规风险）

❌ 人力瓶颈：每个审批人都在做重复的合规检查

智能审批的解法：

申请人用自然语言描述需求 → 专家Agent理解意图+自动核验

→ 数据代理(唯一数据访问归口)：查预算余额、历史同类审批、供应商评级、合同条款

→ 合规官：自动检查政策合规性

→ 验证代理：审计合规判断

→ 主代理：生成审批建议（通过/有条件通过/驳回+理由）

→ 审批人：只需关注例外和建议，不再逐条检查

三级审批机制（→ch10.3双轨执行）：

L1 自动通过（低风险+合规+预算内）→ 固化技能处理，毫秒级响应

例：常规办公用品采购，预算内，供应商白名单

agent自动执行，事后通知审批人

L2 建议审批（中风险+需人工确认）→ 专家推理+人工确认

例：新供应商首次采购，金额在预算内

agent提供审批建议+核验结果，审批人一键确认

L3 完整审批（高风险+需深度审查）→ 多专家深度审查

例：超预算采购、战略级合同、合规存疑

agent提供完整分析+多专家意见，审批人深度审查

专家化价值（→ch4.1）：

处理100次采购审批 → 积累采购模式+常见风险+历史决策

→ 越审越准：第101次自动识别"这个供应商上次延迟交付"

→ 越审越快：相似请求自动归类，L1自动通过比例持续提升

权限归口价值（→ch4.2, ch9.4）：

所有审批数据 → 数据代理唯一入口，脱敏后分发

所有外网核验 → 侦察代理唯一出口
所有合规判定 → 合规官代理统一执行

高稳定执行性 (→ch10.3) :

L1自动通过 → 固化技能, 不依赖大模型推理
故障隔离 → 合规官异常时自动降级为L3 (需人工审查), 不自动通过

权限要点 (→ch5) :

申请人 → 只看到自己的申请+审批进度
审批人 → 看到核验结果+审批建议+关联历史
管理层 → 审批统计+异常模式
审计追溯 → 每次审批的全链路记录不可篡改

11.6 研发效能平台

核心价值: 技术专家agent服务全研发团队, 从架构到代码到部署的知识跨项目积累 (ch4.1专家化)

服务形态: 1:N 专家化 + N:1 权限归口 (→ch4.1, ch4.2)

协作模式: 咨询型 + 项目型 (→ch8.1, ch8.4)

研发团队 → 专家Agent(1:N)
项目A的架构问题 → 积累架构模式
项目B的性能问题 → 积累性能优化知识
项目C的安全漏洞 → 积累安全模式
→ 跨项目洞察: A的架构+B的性能问题=统一优化方案

数据代理(唯一代码仓库入口):

读取代码 → 按项目权限脱敏 (项目B的代码不暴露给项目A团队)
读取CI/CD日志 → 按团队权限

权限要点:

代码访问 → 按项目/仓库粒度控制
部署权限 → 仅DevOps角色可触发
安全扫描结果 → 仅安全团队+项目负责人可见

企业层

11.7 智能投研平台

核心价值: 专家agent同时服务投研/风控/交易三组人, 跨领域关联发现 (ch4.1专家化), 数据代理控制内部持仓数据访问 (ch4.2权限归口)

服务形态：1:N 专家化 + N:N 协作 (→ch4.1, ch4.3)

协作模式：咨询型 + 会诊型 (→ch8.1, ch8.5)

研究员(不同权限) → 专家Agent(1:N, 见多识广)

处理投研问题 → 积累行业知识

处理风控问题 → 积累风险知识

→ 跨领域发现: 某行业趋势+某政策变化=系统性风险

→ 主动推送给相关用户

数据流 (→ch6.2网络分区):

侦察代理(唯一外网出口) → 采集Bloomberg/Reuters/财新

数据代理(唯一数据访问归口) → 查询内部持仓+历史研报 (脱敏后下发)

财经专家 → 分析影响+估值+风险

验证代理 → 交叉检查

权限要点 (→ch5.2, ch5.5):

admin → 完整分析+原始数据+操作建议

trusted → 分析摘要+建议 (数据代理返回脱敏版)

11.8 多群信息分发与专家积累

核心价值：一个专家agent服务多个群，每个群看到不同技能/内容，但agent的知识跨群积累——AI群的论文+财经群的政策=AI监管趋势 (ch4.1专家化)

服务形态：1:N 专家化 (→ch4.1)

协作模式：监控推送型 (→ch8.3)

单 专家Agent 实例服务多个群:

AI日报群(restricted) → 绑定AI技能 → 禁止命令/写操作 → 只读+推送

财经群(restricted) → 绑定财经技能 → 禁止命令/写操作 → 只读+推送

管理群(admin only) → 全部权限 → 可管理技能/配置/数据

开放群 → 问答+搜索 → 无技能绑定 → 无写操作

专家Agent的跨群洞察 (→ch4.1专家化):

AI群讨论"欧盟AI法案" + 财经群讨论"科技股监管风险"

→ 专家Agent发现关联 → 主动在两个群推送: "欧盟AI法案将影响你们讨论的这些公司"

权限要点 (→ch5.3群权限三道防线):

按群权限决定推送内容

按群技能绑定决定用什么视角解读

按用户角色决定推送深度

11.9 能力驾驶舱

核心价值：技能覆盖率/知识缺口/培训需求→组织能力图谱，专家Agent积累所有项目经验发现能力短板（ch4.1专家化）

服务形态：1:N 专家化 + N:N 协作（→ch4.1, ch4.3）

协作模式：咨询型 + 监控推送型（→ch8.1, ch8.3）

能力驾驶舱：

技能覆盖率/知识缺口/培训需求 → 组织能力图谱

专家Agent积累所有项目经验 → 发现能力短板

例："近3个项目都需要LLM微调，但团队只有1人擅长"

权限要点：

HR → 看能力全景+培训需求

管理层 → 看能力汇总+风险（→ch5.5：能力评估数据为internal级别）

11.10 执行驾驶舱

核心价值：任务执行率/阻塞项/SLA达标率→执行力仪表盘，侦察代理监控外部依赖提前预警（ch4.2权限归口）

服务形态：N:N 协作（→ch4.3）

协作模式：监控推送型 + 链式加工型（→ch8.3, ch8.6）

执行驾驶舱：

任务执行率/阻塞项/SLA达标率 → 执行力仪表盘

侦察代理监控外部依赖 → 执行风险提前预警

例："供应商交付延迟概率70%，建议启动备选方案"

权限要点：

PM → 看项目执行详情

管理层 → 看全局执行汇总+异常模式

11.11 专利监控与侵权预警

核心价值：侦察代理是唯一外网出口，数据代理是唯一内部数据访问归口——所有数据访问归口管理，审计链完整（ch4.2权限归口）

服务形态：N:N 协作（→ch4.3）

协作模式：监控推送型 + 链式加工型（→ch8.3, ch8.6）

侦察代理(永久，唯一外网出口) → 持续监控全球专利数据库

→ 发现竞品新专利 → 告警主代理

→ 专利专家：相似度分析+侵权判断

→ 数据代理(唯一数据访问归口)：比对自有专利库（返回脱敏结果）

→ 验证代理：审计分析逻辑

→ 主代理：按权限输出

权限归口价值（→ch9.4）：

侦察代理→外网访问：只有一个出口，所有采集都有审计记录

数据代理→内部数据：只有一个入口，脱敏策略一致

如果分散：10个agent各自访问外网=10个审计点=10个可能泄漏的通道

11.12 安全合规审计

核心价值：所有数据访问通过数据代理归口，审计日志不可篡改，合规检查结果可追溯（ch4.2权限归口 + ch9.3审计系统）

服务形态：N:N 协作（→ch4.3）

协作模式：链式加工型（→ch8.6）

主代理 → 定期触发

数据代理(唯一数据访问归口) → 提取业务操作日志

合规官 → 检查政策合规性

验证代理 → 审计合规官判断（独立于执行链）

侦察代理(唯一外网出口) → 对照最新监管政策

→ 主代理：合规报告+整改建议

审计日志归档（不可篡改，WORM存储）

所有数据访问记录 → 可追溯到具体代理、时间、请求内容

11.13 供应链风控监控

核心价值：侦察代理集中管理外网采集（供应商/物流/政策），数据代理集中管理内部采购数据——归口才能完整风控（ch4.2权限归口）

服务形态：N:N 协作（→ch4.3）

协作模式：监控推送型 + 链式加工型（→ch8.3, ch8.6）

侦察代理(永久，唯一外网出口) → 持续监控：

供应商舆情、物流中断信号、原材料价格波动、地缘政治风险

数据代理(唯一数据访问归口) → 内部数据：

供应商评估记录、采购合同、库存水位、交付历史

风控专家 → 交叉分析外部信号+内部数据

→ 发现：某供应商舆情恶化 + 该供应商是我司核心材料唯一来源

→ 主动告警：供应链断裂风险，建议启动备选供应商

权限要点：

采购部门 → 完整供应商信息+替代方案

生产部门 → 仅影响评估+预计交付延迟

管理层 → 风险汇总+决策建议

11.14 技术架构评审

核心价值：多专家会诊+数据访问归口——专家不直接访问数据，通过数据代理获取脱敏后上下文（ch4.2权限归口）

服务形态：N:1 权限归口 + N:N 协作（→ch4.2, ch4.3）

协作模式：会诊型（→ch8.5）

调度器 → 识别为"技术"主题 → 路由到技术专家 Agent

数据代理 → 获取现有架构+性能数据（脱敏后分发给各专家）

技术专家 → 评估可行性+风险（只看到脱敏数据，不知内部IP/凭证）

侦察代理 → 调研最佳实践+开源方案（唯一外网出口）

合规官 → 检查数据安全+隐私

→ 主代理：评审意见+替代方案+风险矩阵

权限归口价值（→ch9.4）：

专家不直接访问数据源 → 数据代理控制脱敏粒度

侦察代理不直连外网 → 代理层控制出口审计

11.15 跨部门战略决策

核心价值：多专家会诊+权限归口——不同部门看到不同深度的同一份分析（ch4.2权限归口 + ch5.5数据分级）

服务形态：N:N 协作（→ch4.3）

协作模式：会诊型（→ch8.5）

CEO提出战略问题 → 主代理

财经专家：财务影响+ROI（从数据代理获取财务数据，脱敏后分析）

技术专家：技术可行性+成本

合规官：法律风险+监管要求

侦察代理(唯一外网出口)：市场竞品+行业趋势

→ 主代理：识别共识与分歧 → 多视角综合判断

权限要点（同一份分析，不同人看不同深度）：

CEO → 完整分析+原始数据+所有专家意见

部门总监 → 本部门相关深度+其他部门摘要

执行层 → 仅执行所需信息

董事会 → 战略级摘要+关键决策点

11.16 法律文书智能生成与审查

核心价值：专家agent积累合同/判例/法规知识，数据代理管控敏感条款访问（ch4.1专家化 + ch4.2权限归口）

服务形态：1:N 专家化（→ch4.1）

协作模式：项目型 + 链式加工型（→ch8.4, ch8.6）

法务团队 → 专家Agent(1:N)
合同审查 → 积累合同条款模式+风险点
诉讼分析 → 积累判例+策略
合规检查 → 积累法规+违规案例
→ 跨领域洞察：合同条款+最新判例→发现条款风险变化

数据代理(唯一数据访问归口)：

内部合同库 → 脱敏后分发给专家（去除商业机密条款）
客户信息 → 严格脱敏

权限要点：

合同完整版 → 仅承办法务+审批人可见
合同摘要 → 团队可见（去除金额/条款细节）
客户身份 → PII级别，跨代理传输必脱敏

11.17 PPTX 智能生成

核心价值：调度器按主题路由到专业Agent，数据代理控制数据访问范围（ch4.2权限归口）

服务形态：N:1 权限归口（→ch4.2）

协作模式：项目型（→ch8.4）

调度器 → 识别为"创造"主题 → 路由到创作 Agent
数据代理 → 获取数据+模板（按用户权限脱敏）
执行代理A → 生成大纲+内容
执行代理B → 设计图表+排版
验证代理 → 检查数据准确性+脱敏完整性
→ 主代理：整合+格式转换+投递

权限要点：

执行代理不直接访问数据 → 通过主代理注入脱敏上下文
生成的PPT不含PII → 验证代理检查脱敏
高管版 vs 公开版 → 不同脱敏级别输出

11.18 智能客服平台

核心价值：一个专家agent处理所有客服问题，从售前到售后到技术支持，知识跨场景积累（ch4.1专家化）

服务形态：1:N 专家化 + N:1 权限归口 (→ch4.1, ch4.2)

协作模式：咨询型 + 调度型 (→ch8.1, ch8.2)

客户 → 权限归口调度器 → 专家Agent
售前咨询 → 积累产品知识+常见顾虑
技术支持 → 积累故障模式+解决方案
投诉处理 → 积累客诉模式+根因分析
→ 跨场景洞察：
技术支持发现某故障高频 → 主动建议售前团队告知客户
投诉发现某功能引发不满 → 主动建议产品团队优化

权限要点：

客户 → 只看到自己订单/产品相关信息
客服 → 看到完整订单+技术细节，但看不到其他客户数据
管理层 → 看到汇总统计+趋势分析，不暴露单客户信息

11.19 企业知识助手

核心价值：一个专家agent服务全公司，从HR到财务到技术的问题都处理，积累跨部门知识 (ch4.1专家化)

服务形态：1:N 专家化 (→ch4.1)

协作模式：咨询型 (→ch8.1)

全公司员工 → 专家Agent(1:N)
HR问："年假政策是什么？" → 积累HR知识
财务问："报销流程？" → 积累财务知识
技术问："VPN怎么连？" → 积累IT知识
→ 跨部门洞察：
新员工问"入职需要办什么" → 专家整合HR+IT+财务的回答
比三个独立agent分别回答更连贯

权限要点：

每个员工只看到自己有权限的信息
HR问答中的薪酬数据 → 对非HR脱敏
财务数据 → 仅财务部门可见完整版

11.20 知识体系持续构建

核心价值：专家agent积累跨领域知识——AI论文+财经政策+专利趋势→跨领域知识图谱 (ch4.1专家化)

服务形态：1:N 专家化 + N:N 协作 (→ch4.1, ch4.3)

协作模式：监控推送 + 链式加工 (→ch8.3, ch8.6)

侦察代理(永久) → 持续采集arxiv/GitHub/博客
 → 数据代理 → 清洗+去重+结构化
 → 知识专家(1:N, 服务所有知识需求) → 提取知识骨架+关联+演进
 → 跨领域发现: AI论文中的技术→财经群的政策影响→专利群的专利趋势
 → 验证代理 → 交叉验证一致性
 → 主代理 → 知识库更新+变更通知

12. 与现有架构的差距与演进路径

12.1 差距分析

能力	现状	目标	差距
用户权限	✅ admin/trusted/blacklist + restricted_channels	完整RBAC + ABAC	需动态策略
群权限	✅ 三道防线(命令/工具/提示词过滤)	群级完整权限矩阵	需细化工具级权限
技能绑定	✅ channel_skill_bindings (Discord/Slack/Feishu)	所有平台支持	需扩展到 Telegram 等
代理间通信	进程内函数调用	跨网络消息总线	需新建通信层
代理调度	smart_model_routing (仅复杂度)	多维度路由+权限归口	需主题/技能路由+归口层
代理生命周期	临时(delegate_task)	永久+临时	需常驻进程管理
状态持久化	无状态子代理	专家有领域知识库	需知识库层
安全分区	单机无分区	核心区/DMZ隔离	需网络架构
主动推送	无	侦察代理主动上报	需告警通道
审计追溯	无	全链路不可篡改审计	需审计系统
数据脱敏	无	按请求方等级脱敏	需脱敏引擎

能力	现状	目标	差距
权限热加载	部分支持(命令/工具实时)	全量热加载	需策略引擎
固化技能执行	无	毫秒级确定性执行	需固化技能引擎+技能定义语言
驾驶舱	无	五种驾驶舱(个人/团队/项目/能力/执行)	需驾驶舱UI+数据汇聚层
智能审批	无	三级审批机制(L1自动/L2建议/L3深度)	需审批流程引擎+合规规则库
故障隔离	单进程，一挂全挂	代理间故障隔离	需多进程架构+降级策略

12.2 演进路径

Phase 1: 权限完善 + 单机多进程（MVP）

目标：完善现有权限体系 + 验证多代理协作可行性

- 工具级权限（denied_tools per channel）
- 主代理 + 侦察代理独立进程
- Redis Stream 单机通信
- 固化技能引擎（定时任务+规则匹配）
- Docker Compose 部署

Phase 2: 网络分区 + 权限归口层

目标：核心区/DMZ 分离 + 权限归口调度

- 侦察代理部署到 DMZ
- mTLS 双向认证
- 权限归口调度器（鉴权+路由+审计）
- 数据脱敏引擎
- L1自动审批（固化技能）

Phase 3: 专家体系 + N:N 协作 + 驾驶舱

目标：永久专家代理 + 消息总线 + 驾驶舱

- 专家代理知识库 + 自我改进
- 消息总线（Redis Stream → NATS）
- 编排引擎

- 服务注册/发现
- 审计系统
- 五种驾驶舱（个人/团队/项目/能力/执行）
- L2/L3智能审批

Phase 4：生产就绪

目标：企业级可靠性

- 高可用（健康检查 + 自动重启 + 负载均衡）
- 故障隔离（任一代理异常不影响其他代理）
- 全链路追踪（OpenTelemetry）
- Kubernetes 部署
- 监控告警（Prometheus + Grafana）
- 固化技能↔专家推理双向升级

13. 附录A：消息协议定义

JSON-RPC 2.0 方法清单

方法	方向	通道	说明
task.execute	主→子	task.*	下发任务
task.cancel	主→子	task.*	取消任务
result.submit	子→主	result.*	提交结果
result.stream	子→主	result.*	流式提交
alert.signal	子→主	alert.*	主动告警
alert.ack	主→子	alert.*	确认告警
hb.ping	子→主	hb.*	心跳
hb.pong	主→子	hb.*	心跳确认
audit.log	所有→审计	audit.*	审计日志
config.update	主→子	task.*	配置热更新
knowledge.sync	专家↔知识库	task.*	知识同步
permission.check	子→主	task.*	权限校验请求

方法	方向	通道	说明
permission.grant	主→子	result.*	临时授权

错误码

码	含义
-32700	JSON 解析错误
-32600	无效请求
-32601	方法不存在
-32602	参数无效
-32603	内部错误
-32001	代理不可用
-32002	任务超时
-32003	权限不足
-32004	数据脱敏失败
-32005	知识库不一致
-32006	令牌过期
-32007	证书无效
-32008	群权限拒绝
-32009	用户角色不足
-32010	数据分级冲突

14. 附录B：权限策略配置参考

完整群权限配置

```
gateway:
  user_roles:
    admin:
      - ou_d366ba075912a3865430a7fe7020454b
  trusted:
```

```

- ou_trusted_user_1
- ou_trusted_user_2

restricted_channels:
  oc_6ed1cc645057da3bbe93c6120219a61e: # AI日报群
    users:
      - id: ou_d366ba075912a3865430a7fe7020454b
        role: admin
      - id: ou_trusted_user_1
        role: trusted
    allowed_commands: [] # 禁止所有命令
    allowed_toolsets: # 允许的工具集
      - web
      - file
    denied_tools: # 禁止的工具
      - terminal
      - write_file
      - skill_manage
      - memory
    max_iterations_per_user: 30 # 用户级迭代预算
    blacklist:
      - ou_blacklisted_user

feishu:
  channel_skill_bindings:
    - id: "oc_6ed1cc645057da3bbe93c6120219a61e"
      skills: ["group-thinking", "source-evolution", "core-thinking"]
  channel_prompts:
    "oc_6ed1cc645057da3bbe93c6120219a61e": |
      你是AI日报群的助手。
       允许：回答问题、搜索信息、推送日报
       禁止：修改技能、执行命令、有副作用的操作

```

代理信任等级配置

```

agents:
  brain:
    trust_level: L0
    network_zone: core
    allowed_outbound: [vault, expert, scout, worker, auditor]

  scout:
    trust_level: L3
    network_zone: dmz
    allowed_outbound: [brain] # 只能回传主代理
    allowed_external: true # 允许外网访问

```

```
max_result_size_kb: 512
```

```
vault:
```

```
  trust_level: L1
```

```
  network_zone: core
```

```
  allowed_outbound: [brain] # 只能返回主代理
```

```
  data_access: read_only
```

```
  desensitization: true      # 返回结果需脱敏
```

```
expert:
```

```
  trust_level: L2
```

```
  network_zone: core
```

```
  allowed_outbound: [brain] # 只能返回主代理
```

```
  direct_data_access: false # 不直接访问数据
```

```
  direct_external: false    # 不直接访问外网
```